REPLACING REPOSITORY EXPLORATION WITH TYPED JEV DECISIONS
CS329Z MIDWAY REPORT DRAFT

Kristjan Dagur Egilsson **Ómar Ingi Halldórsson** **Rami Ratl Mrad**

1 RESEARCH QUESTIONS

Our project asks whether a structured decision model can replace a narrow component of an existing coding agent at lower total cost while preserving bug-repair performance. Following the instructor's recommendation to obtain mechanical evidence before redesigning an agent harness, we study OpenCode's repository-exploration subagent. Our intervention uses Jev to select relevant source files from deterministic search candidates; the same strong generative model continues to write and test patches. We compare this intervention against the native generative explorer and a search-only replacement. The primary outcomes are independently tested repair success and total inference cost. The midpoint objective is to establish a working substitution and an authentic evaluation pipeline, not to claim calibrated confidence or performance equivalence from a small development sample.

2 ENVIRONMENT & DATA

Existing agent harness. We use the pinned OpenCode 1.19.5 command-line harness and retain its native build agent, `task` delegation tool, `explore` subagent, file tools, and patch-writing loop (OpenCode contributors, 2026). Each rollout starts from an exact upstream repository snapshot. Its issue prompt asks the build agent to delegate initial localization to the existing explorer before editing, then implement a minimal fix and optionally run existing tests. This prompt is identical across conditions. The parent can recover from imperfect localization using its usual read/search tools. It may edit implementation files under `pydicom/`, but not tests or files outside the repository through the configured tool permissions. These permissions are not an operating-system sandbox.

Authentic development tasks. We use all five pydicom issues in the official SWE-bench Lite development split (Jimenez et al., 2024; SWE-bench contributors, 2026): instances ending in 1139, 1256, 1413, 1694, and 901. The dataset revision, task IDs, and upstream base commits were frozen before agent outcomes. We selected this repository for a locally runnable pure-Python environment; this convenience sample is neither representative of SWE-bench nor an independent final test set. The tasks concern person-name iteration, nested JSON bulk-data handling, byte-valued data elements, suppressed JSON conversion errors, and logging configuration. No invented programming tasks enter the corrected pilot.

The dataset supplies issue descriptions, upstream fix patches, added regression-test patches, and fail-to-pass/pass-to-pass test identifiers. Only the issue and original repository are available to the coding agent. The evaluator's added tests, gold fix, and expected outcomes remain outside the agent workspace and Jev state. We do not supply future test feedback to Jev or change the protocol after observing failed repairs. Public benchmark issues may be present in model training data; we make no contamination-free claim.

Independent local evaluation. After each rollout, the evaluator applies the generated implementation patch and the benchmark's test patch to a separate fresh base snapshot. It executes the supplied fail-to-pass and pass-to-pass selections under Python 3.12 with pinned pytest 7.4.4, setuptools 70.3.0, NumPy 1.26.4, and Pillow 11.3.0. Success requires zero test failures/errors/skips, the expected minimum collection, successful agent termination, and no forbidden tracked-file changes. Every environment was checked before paid agent runs: its broken base fails assertions and its published upstream fix passes. The five controls execute 41, 23, 304, 27, and 5 cases respectively. These are test cases within five issues, not hundreds of independent tasks.

Two compatibility details matter for reproduction. Historical tests use nose-style setup methods retained by pytest 7 but removed in pytest 8. Three upstream parameterized identifiers for instance 1413 are truncated; we run the entire corresponding parent test functions, adding cases instead of silently dropping regressions. Native local pytest differs from SWE-bench's official containerized evaluation. We therefore report *local benchmark-regression success*, not an official SWE-bench resolved score. Official container reproduction remains required for the final study.

3 METHODS

One internal substitution. The build agent uses `openai/gpt-6.1-sol` in every condition. A thin OpenAI-compatible provider adapter exposes distinct build and explore URLs. OpenCode's native `task` tool still creates an explorer session and returns its result to the parent. In the native condition, both provider URLs forward to the same strong model. In the Jev condition, only an explore-provider request is replaced with deterministic retrieval, one typed Jev request, and a formatted list of selected file paths/snippets. There is no additional generative prescreen or newly written agent execution loop.

Candidate construction and Jev decisions. We index Python implementation files, excluding the repository's tests. A deterministic BM25-style search ranks files using the parent agent's actual exploration request. The top 12 candidates contain relative paths and original source snippets: the most query-matching 16-line window in each file, capped at 1,800 characters. This construction uses no gold labels. Jev 1.14 receives the request and candidates and answers one Noul file-relevance question per candidate in a single API call (OpenRouter, 2026). We validate every returned probability, rank files by score, and return the top three paths/snippets to the native explorer session. Thus, Jev determines a concrete internal context-selection decision; it neither generates a patch nor selects the coding model at task launch.

An API error, malformed response, or invalid score falls back to the ordinary generative explorer, counting both the attempted decision and fallback costs. This pilot ranks candidates without a confidence threshold. Noul relevance scores are not calibrated probabilities of patch correctness. A future confidence-based escalation policy requires independent development/calibration data; it cannot be justified by selecting a threshold on these observed failures.

Comparisons and fixed limits. We run three conditions on the same five issues, one rollout each: (1) native generative exploration; (2) retrieval plus Jev selection; and (3) search-only, returning the first three retrieval candidates without Jev. The search-only control asks whether any improvement requires the decision model. The native explorer has whole-repository tool access, whereas both replacements use bounded candidates; this is a component replacement comparison, not identical-context ranker isolation. Parent trajectories are stochastic, and the parent may recover using later searches. A final study should also compare a generative ranker receiving the same candidates and use repeated rollouts where affordable.

All conditions share the issue prompt, strong writer, tools, evaluator, 16 parent steps, 6 native explorer steps, at most 32 model requests, 4,096 output tokens/request, and 480 seconds/run. We execute the native baseline before activating Jev. Task outcomes are paired by issue, but the small sample does not support a noninferiority claim. We report every declared rollout, including unsuccessful repairs.

Accounting and artifacts. The adapter records actual upstream model responses and billed usage for writer, auxiliary, exploration, and Jev calls. A shared durable ledger reserves conservative costs before requests, settles known bills, and retains unresolved reservations. Caps are $2 per rollout and $15 across this pilot. The real upstream key stays in the host adapter; the coding process receives only a temporary local-adapter credential. Each immutable run includes a source/protocol hash, task/base metadata, event trace, request/response receipts, file decisions, generated patch, independent test log, and result. Reusing a matching completed run ID retrieves its stored result rather than spending again. All 15 runs use the same source hash; all bills were captured.

Table 1: Real pydicom development pilot; one rollout per issue/condition. Cost includes all captured model calls. Time is observed agent wall time through the common local adapter, excluding separate evaluator time. Local regression outcomes are not official Docker SWE-bench scores.

Exploration condition	Repairs passed	Mean cost (USD)	Mean time (s)
Native generative explorer	2/5	0.110382	37.64
Jev file selection	2/5	0.072762	25.93
Search only	2/5	0.062014	24.90

4 Summary of Progress

Working prototype and measured comparison. The internal substitution is implemented and exercised on real repository repairs. Table 1 summarizes all 15 development rollouts. Both the native and Jev conditions solved 2/5 issues. Mean total cost changed from $0.110382 to $0.072762, a 34.1% reduction for this sample. Mean agent wall time changed from 37.64 to 25.93 seconds. The five Jev runs used five real decision calls and zero fallback explorer generations, replacing 23 native explorer generation calls. These measurements establish mechanical feasibility; matching observed success on five development issues does not establish equivalent performance.

What the control and failures tell us. The search-only condition solved 2/5 issues with mean cost $0.062014. This control must be considered before attributing a benefit to Jev: bypassing an iterative explorer and providing retrieved context may explain much of the effect. The current experiment is insufficient to establish that Jev is preferable to deterministic retrieval. For example, task 1139's patches implement iteration/containment but fail the benchmark's direct `next(person_name)` requirement. Task 1413's repairs do not handle every byte-valued data-element variant. Task 901 changes logging configuration but fails the benchmark's handler/debug compatibility expectations. These are observable repair failures, not evaluator infrastructure errors: the corresponding gold fixes pass. Imperfect localization can also be hidden by the strong parent's later recovery, so file-selection quality and end-to-end success should be analyzed separately in a broader study.

Completed work and next steps. We have completed dataset/environment controls, native baseline execution, the Jev integration, all three pilot conditions, immutable metering/trace artifacts, and four new checks covering malformed decisions, gold/test leakage in retrieval, fallback, and budget denial. Together with the existing checks, 15 offline tests pass. New work is isolated on branch `rami`, and the README records runnable commands, results, ownership, remaining work, and cost. This corrected pilot cost $1.225791 OpenRouter; no Tinker credits were used. The earlier synthetic pilot is preserved as historical integration work and excluded from these results.

The immediate submission tasks are independent teammate reproduction, review of this draft and AI disclosure, confirming repository grader access, and portal submission. Video preparation is deferred at the user's request. For the final study, we will reproduce with the official evaluator, broaden repositories and development tasks, examine retrieval/selection errors and parent recovery, and freeze independent evaluation settings before inspecting final outcomes. We will report paired uncertainty, costs, latency, and failure categories rather than extrapolating this sample. The original BANKING77/CLINC150 intent-decision and tool-using escalation proposal remains a deferred option. Training with the team's separate Tinker accounts is conditional on a demonstrated need and sufficient separately sourced data; it is not a prerequisite for this experiment.

Safety considerations. We use public software issues and repository fixtures, not clinical patient data or deployed medical systems. Generated patches are research artifacts and are not merged upstream or applied in production. Credentials are excluded from published traces. Future evaluation environments should use container/process isolation in addition to tool-level permissions, retain upstream license notices, and record actual resource cost.

REFERENCES

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? *International Conference on Learning Representations*. https://arxiv.org/abs/2310.06770.

SWE-bench contributors. SWE-bench Lite dataset, development split. Revision 6ec7bb89b9342f664a54a6e0a6ea6501d3437cc2. Accessed November 6, 2026. https://huggingface.co/datasets/princeton-nlp/SWE-bench_Lite.

OpenCode contributors. OpenCode 1.19.5, native task/explore implementation. Software source. Accessed November 6, 2026. https://github.com/anomalyco/opencode/blob/v1.19.5/packages/opencode/src/tool/task.ts.

OpenRouter. Jev documentation: using the TypeSafe decision model on OpenRouter. API documentation. Accessed November 6, 2026. https://openrouter.ai/docs/guides/community/jev.

AI-USE DISCLOSURE STATEMENT

OpenAI Codex assisted Rami with interpreting the instructor feedback, inspecting source/documentation, implementing the provider adapter and evaluator, executing the model experiments, and drafting this report. Jev 1.14 made file-relevance decisions within the evaluated agent; GPT-6.1-Sol produced the experiment's patches and native exploration responses. Benchmark issues, upstream fixes, and regression tests came from SWE-bench/pydicom, rather than being generated for this project. This is an unsubmitted draft requiring team review and confirmation against the course disclosure policy; it does not represent completed independent review by every author.